

Imputation of missing pixel art character poses with differentiable palette quantization[☆]

Flávio Coutinho^{a,b}, Lucas G.S. Chaves^a, Luiz Chaimowicz^a

^a Universidade Federal de Minas Gerais, Belo Horizonte, MG, Brazil

^b Centro Federal de Educação Tecnológica de Minas Gerais, Belo Horizonte, MG, Brazil

ARTICLE INFO

Dataset link: <https://huggingface.co/datasets/puckquiere/pac>, <https://github.com/fegemo/multi-domain>

Keywords:

Generative adversarial network

Procedural content generation

Missing data imputation

Character sprite

Pixel art generation

Palette quantization

ABSTRACT

Designing pixel art character sprites with numerous animation frames is a labor-intensive process that often involves repetitive work. To streamline this task, we propose a method that automates sprite generation, allowing artists to focus on the creative aspects. Our work addresses the challenge of synthesizing a character sprite in a target pose using reference images from other viewing angles. We formulate this as a missing data imputation problem and introduce a generative adversarial network that reconstructs the desired pose from those already available among the back, left, front, and right directions. Unlike baseline models, our proposed generator utilizes all available poses of a character to enhance the quality of the generated image. Additionally, it does not introduce small variations of the same colors and instead produces images that strictly follow a predefined color palette. Crucially, our model ensures adherence to the palette by incorporating a novel operation of differentiable quantization of pixel values, making it suitable for end-to-end training. Compared to baseline models proposed for generating a new pose from a single one, our approach produces images with better FID (34.09% lower) and L_1 distance to the ground truth (22.66% lower). It also shows superior quality through visual inspection. Additionally, as the generator selects colors from a desired palette, similar to how human artists create pixel art, the generated images are more readily useful, eliminating the need for a post-processing step to restrict the color.

1. Introduction

Asset creation is a critical and time-consuming phase in game development that often requires significant portions of the timeline of a project. Character design, in particular, is rarely a linear process and frequently requires iterative revisions [1]. This challenge is especially pronounced in pixel art games, where the color of each pixel is carefully chosen, and the number of sprites grow with the amount of animations and viewing angles [2].

Although character creation inherently relies on artistic creativity, certain tasks within the pipeline can become repetitive. Examples include generating normal maps from colored sprites [3], designing individual animation frames [4], or propagating edits across a character's many sprite poses and facing directions. Recent advances in Procedural Content Generation via Machine Learning (PCGML) offer promising solutions to streamline these workflows. Prior work has explored character generation using PCGML techniques such as Variational Autoencoders (VAEs) [5,6], Generative Adversarial Networks

(GANs) [4,7–10], and Convolutional Neural Networks (CNNs) [11]. These approaches typically frame the problem as image-to-image translation, where one image (e.g., a character facing left) is transformed into another (e.g., the character facing front) [4,8]. However, leveraging additional available data, such as multiple reference poses, could improve the output quality.

In a previous work [12], we addressed the problem of generating a character sprite in a target pose by treating it as a missing data imputation task. Our proposed model synthesizes the missing pose (e.g., a front-facing view) using the available sprites of a character in other orientations (e.g., left, right, or back).

We introduced a GAN architecture¹ based on CollaGAN [13], with adjustments to the generator's topology and training procedure. By evaluating the model using the metrics Fréchet Inception Distance (FID) [14] and L_1 distance, we showed that our model matches or surpasses state-of-the-art performance, particularly when more poses of a character are available. While fewer inputs result in lower-quality

[☆] This article is part of a Special issue entitled: 'SBGames 2024' published in Entertainment Computing.

^{*} Corresponding author at: Centro Federal de Educação Tecnológica de Minas Gerais, Belo Horizonte, MG, Brazil.

E-mail addresses: fegemo@cefetmg.br (F. Coutinho), lucassgc@ufmg.br (L.G.S. Chaves), chaimo@dcc.ufmg.br (L. Chaimowicz).

¹ Source code: <https://github.com/fegemo/multi-domain>.

outputs, the model remains robust. An ablation study demonstrates the impact of each architectural modification made to the original CollaGAN on the final results. One remaining challenge was that due to the nature of the neural network working in the continuous space, the generated images usually contain many tiny variations of the same color, which is well suited for generating photorealistic images but not for pixel art.

Hence, in this extended paper, we introduce changes that ensure that the generated images conform to their intended palettes – a fundamental requirement for pixel art consistency. Quantizing the generated image to some palette has been used in a post-processing step [8]. However, we show that embedding the quantization inside the generative model not only improves the quality of the generated images, but also eliminates the need for a multi-stage pipeline.

A challenge with replacing pixels of the generated image with the closest match from a palette is that the typically used operations do not have defined computable gradients. Hence, to allow gradient-based optimization of the network, we devised a *Differentiable Palette Quantization* operation, which enables training models end-to-end to produce pixel art.

In addition, we found it beneficial to include a loss term in the generator's objective that steers it towards using all of the palette colors: the *Palette Coverage Loss*. Otherwise, some palette colors with a very low frequency (e.g., the eye colors) might not be used.

Our key contributions are:

- A unified GAN framework capable of generating multiple character poses using a single generator-discriminator pair while receiving a variable number of images as input;
- Architectural refinements to CollaGAN that improve the quality of the output images;
- Generation of images whose colors are constrained by an intended palette;
- Introduction of a palette quantization operation that is differentiable, and hence, can be used in gradient-based optimization;
- Proposition of the palette coverage loss to steer the generation towards using all available colors.

2. Related work

We first present some works that use deep generative models to create characters, emphasizing those that use the pixel art style. In sequence, we survey different approaches for handling pixel art, including the challenges and how they have been tackled.

2.1. Character generation

Advances in generative models have given rise to data-centric approaches to creating game development assets. We are particularly interested in techniques that could at least partially automate artistic tasks that are more repetitive than creative, such as drawing different viewing angles of characters or in-between animation frames.

Tackling the generation of shaded and segmented sprites from line-art sketches of characters, Serpa and Rodrigues [9] initially used conditional GANs in an image-to-image translation task before reformulating it as semantic segmentation with specialized losses [11]. Rather than using adversarial training as in the former approach, the second model employed dense connections to increase the network's depth, implemented deep supervision for better gradient propagation, and applied a class-weighted focal loss to mitigate class imbalance issues.

Also related to shading of pixel art characters, Nikolov [15] introduced a U-net [16] that creates a light map from a colored sprite and the information of a light position. It uses conditional instance normalization layers to inject the light position at different stages of the network. The generated maps are usually contained within the character silhouette but sometimes contain holes and do not accurately

capture the three-dimensional geometry of the source character image. Additionally, the generated light maps contain an unbounded number of shades of gray, which does not conform well to the pixel art style.

To generate new Pokémon characters, Saravanan and Guzdial [6] trained a Vector-Quantized VAE [17] on sprites of known Pokémon, allowing the model to learn a discrete latent representation. Later, they trained a PixelCNN [18] to create new sprites resembling the training distribution.

Animation interpolation has also been investigated as a related problem. Loftsdottir and Guzdial [5] proposed SketchBetween, which generates intermediate animation frames given start/end poses and rough sketches of the middle frames. The model produces plausible transitions for poses similar to those seen during training, though with some blurring characteristic of VAE-based approaches.

Some authors investigated creating characters from text prompts. Löwenström [19] proposed an optimization process that transforms a text prompt and an initial image into a new one. They achieve the text-and-image translation through a pre-trained generative model and a model that performs a text-similarity loss. The resulting images conform to the prompt, in general, but suffer from the use of many color variations and blurriness. With a similar objective, Mercioni et al. [20] use a VQGAN [21] with a text-similarity loss to produce variations of an input sprite. Although the model attempts to produce images in the pixel art style as requested by the prompt, they do not adhere to the palette of the input image, using many more colors than intended for the style.

Some works explored character generation in target poses by conditioning the generation on bone-graph representations. Hong et al. [7] developed a Multi-Discriminator GAN (MDGAN) that transforms a sprite of character in a neutral pose into a target pose by conditioning on a target bone structure image, using separate discriminators for shape consistency and pose accuracy. Similarly, Choi et al. [10] created a motion dataset by processing real video frames through body segmentation networks, enabling pose-conditioned character generation. While effective, these methods require carefully aligned training data that limits their adaptability to diverse character types and movements.

Also targeting pose translation, Coutinho and Chaimowicz [8] followed a different strategy of generating images of characters in a fixed target pose from images of them facing a source one in an image-to-image translation task. Even though the model generated potentially useful images, their architecture could translate images only between two domains and in a single direction. The authors evaluated their architecture training on small and stylistically cohesive datasets (less than 1,000 examples) compared to a larger and more diverse one (approximately 12,000 examples) that contained the smaller datasets and a greater variety of miscellaneous character sprites. They observed improved results by using more examples.

Our approach builds upon these foundations in character pose generation (cf. [7,10]) and pose translation (cf. [8]) but introduces a key innovation: we frame the problem as a missing data imputation task. Rather than learning pairwise pose translations, our model synthesizes missing poses by reasoning about all available views of a character simultaneously, enabling more flexible and generalizable generation. Compared to Coutinho and Chaimowicz [8], our proposed model is a multi-domain generator, as it can produce images for all domains (one at a time), whereas [8] requires training a separate model for each translation direction.

2.2. Pixel art generation

Although the generation of pixel art may appear less complex than that of photorealistic images, certain distinctive characteristics of this art style introduce specific challenges. These include using a limited color palette, a highly discrete color distribution, and applying specialized color blending techniques such as dithering and color ramps [22].

These challenges have been addressed in the literature through various approaches.

Saravanan and Guzdial [6] enhanced pixel art quality by modifying a Vector-Quantized VAE architecture [17], adding paired 1×1 convolutional layers at both the encoder and decoder stages. This modification, tested on Pokémon sprites, effectively reduced the blurriness typical of VAE outputs while maintaining the crisp edges characteristic of pixel art. However, the generated images still do not resemble real characters, presenting spurious blobs of pixels and high-frequency color noise.

Mercioni et al. [20] assessed the generation of pixel art sprites of objects and animals from a combination of text prompts and input images. To enhance the adherence to the pixel art style, the authors suggested a post-process step to manually remove the unwanted background, as the underlying model does not accept an alpha channel, and to reduce the noise present in the generated images.

Coutinho and Chaimowicz [22] investigated two hypotheses to improve the quality of the generated pixel art: (a) representing the pixels of each image as indices in their color palettes to enforce the generated images to only use colors from their palettes, and (b) adding a histogram loss term to the generator to steer it towards making images with the same histogram of the input. While the former led to strong overfitting, the latter yielded marginally better results, but the images still present many small variations of the same colors. The authors also highlighted the potential benefits of having a higher number of examples to train with as a crucial means to enhance the generalization capacity of their model.

Wu et al. [23] proposed an architecture based on CycleGAN [24] to translate real images into the pixel art style and vice-versa. To improve the quality of the generated images, they pre-trained a U-net to remove anti-aliasing effects, making the edges sharper. It processes the intermediate output of the network, effectively acting as a post-process step. The pipeline makes the images much closer to the desired pixel art style, even though it still introduces some small color variation, as there is no hard constraint on color usage.

In a later work, Coutinho and Chaimowicz [8] employed a post-processing step to enforce palette adherence in their GAN-generated images, wherein each pixel was quantized to the nearest color in a predefined palette. The authors evaluated several color spaces for computing perceptual distances between colors, determining that the CIELAB space yielded the best results. While this approach guarantees strict palette compliance – an essential feature for the generation of pixel art – it provides only marginal improvements in the perceptual quality of the generated images.

The challenge to embed the palette quantization inside a model that uses gradient-based optimization is that it involves non-differentiable operations, such as picking the palette color with the minimum distance (e.g., tf.argmax) or sampling the logit with the highest value (e.g., tf.argmax). Similar problems have been approached using the Gumbel-Softmax reparameterization trick [25], which allows sampling from categorical distributions in a way amenable to differentiation. Using that technique, Binniger and Sorkine-Hornung [26] introduced SD- π XL, a method based on score distillation sampling that uses an off-the-shelf diffusion model and is optimized to generate pixel art from a text prompt, a color palette, and an optional source image. The proposed model can create images that follow a specified palette using soft or hard constraints. In the former, the resulting images can use colors that lie inside the convex hull of the color palette, which usually results in many more colors than intended. In contrast, in the latter, the generated images only use colors from their palettes. Even though the method yields high-quality pixel art images, as it involves optimizing the input image (i.e., it is not just a feed-forward pass of a GAN's generator), it takes a long time to generate a single image (e.g., hours).

In this extended paper, we propose a palette quantization operation that is friendly to differentiation similar to how SD- π XL [26] used the Gumbel-Softmax reparameterization [25]. However, we resort to an annealed softmax with temperature operation, as explained in Section 6 (Differentiable Palette Quantization).

3. Architecture

In our previous work [8], we propose an architecture based on ColLaGAN [13] to impute images of pixel art characters in a missing pose (target domain). In this extended paper, we introduce some changes to the generator topology and training to ensure the generated images strictly adhere to the specified color palette. Such changes are presented in Section 6 (Differentiable Palette Quantization).

To facilitate understanding our proposed model, let us consider that there are $N = \{a, b, c, d\}$ domains, one representing each pose. The architecture consists of a single generator and discriminator pair, of which the former creates an image $\hat{x}_t$ of a character in the missing pose t using the available images from all of the other source S poses:

$$\hat{x}_t = G(x_S, t), \text{ with } t \in N, S = N - \{t\}$$

Our generator has one encoder branch to process the input from each domain, a single decoder branch with concatenated skip connections, and outputs an image in the missing domain. The discriminator distinguishes images as real or fake and determines their domain through an auxiliary classifier output. Fig. 1 shows the topology of both networks. In this extended paper, the diagram includes an additional input of the desired palette and the palette quantization layer as the last operation of the generator.

3.1. Objective function

We optimize both networks adversarially while incorporating additional objectives. The generator's loss consists of five components: regression, cycle consistency, structural similarity, adversarial, and domain classification losses. Meanwhile, the discriminator is trained using adversarial and domain classification objectives.

Each training step consists of a forward and a backward pass. In the forward one, the generator G receives a minibatch of paired images with a random missing domain t , synthesizing an image corresponding to t . For example, if $S = \{a, b, c\}$ and $t = d$, the images x_a, x_b, x_c are available and we want the model to generate $\hat{x}_d$ as close as possible to the real x_d :

$$\hat{x}_d = G(\{x_a, x_b, x_c, x_{\text{zero}}\}, d),$$

in which x_{zero} is a tensor filled with zeros with the same dimension of an input image.

Subsequently, to ensure cycle consistency, the backward pass synthesizes $|N| - 1$ images with each domain in $S = \{a, b, c\}$ as a target, using the generated $\hat{x}_d$ instead of the real x_d . In our example, the outputs of this pass would be:

$$\tilde{x}_{a|d} = G(\{x_{\text{zero}}, x_b, x_c, \hat{x}_d\}, a)$$

$$\tilde{x}_{b|d} = G(\{x_a, x_{\text{zero}}, x_c, \hat{x}_d\}, b)$$

$$\tilde{x}_{c|d} = G(\{x_a, x_b, x_{\text{zero}}, \hat{x}_d\}, c),$$

and should reconstruct the original images x_a, x_b , and x_c .

A regressive loss term $\mathcal{L}_{\text{reg}}$ steers the generator towards using the information from the source domains to translate an image to the target, whereas a multiple cycle consistency loss $\mathcal{L}_{\text{mcy}}$ leads it into encoding in $\hat{x}_t$ enough information to allow cyclical reconstruction of the original inputs. Both losses are pixel-wise L_1 distances between the real and generated images:

$$\mathcal{L}_{\text{reg}} = \mathbb{E}_{x_t, \hat{x}_t} [\|x_t - \hat{x}_t\|_1]$$

$$\mathcal{L}_{\text{mcy}} = \mathbb{E}_{x_s, \tilde{x}_{s|t}} \left[\sum_{s \in S} \|x_s - \tilde{x}_{s|t}\|_1 \right]$$

Besides $\mathcal{L}_{\text{mcy}}$, an additional objective $\mathcal{L}_{\text{sim}}$ is used to improve the quality of the images generated in the backward pass. It uses the structural similarity index measure (SSIM) [27] to compose a loss term between the cyclically generated $\tilde{x}_s, \forall s \in S$ and the real source images

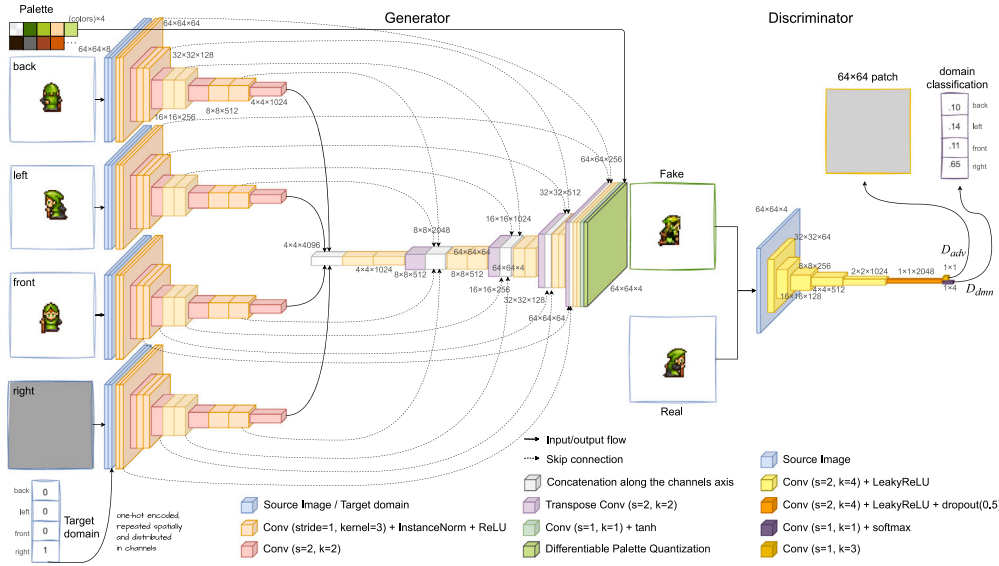


Fig. 1. Architecture of the proposed model. **LEFT:** The generator receives a character in the source domains, a palette, and a label indicating the target, which is one-hot encoded, spatially spread, and concatenated with each input image. The inputs follow the encoder branches and are concatenated at the bottleneck layer, flowing into the unified decoder. Skip connections provide early outputs to the decoder. **RIGHT:** The discriminator receives the image (real or fake) that must be distinguished and outputs D_{adv} with the real/fake logit and D_{dmn} with the probabilities of the image being part of each domain.

x_s . Its formulation is given by the following equation, where $P(x_1, x_2)$ is the set of corresponding pixel pairs between two images, $|P|$ is its cardinality, and $SSIM(p)$ is the measure between them:

$$\mathcal{L}_{ssim} = \mathbb{E}_{x_s, \tilde{x}_{s|t}} \left[-\log \left(\frac{1}{2|P|} \sum_{p \in P(x_s, \tilde{x}_{s|t}), s \in S} (1 + SSIM(p)) \right) \right]$$

The next two terms are used by the generator and the discriminator: adversarial and domain classification. The adversarial loss uses the one from Least Squares GAN [28], which optimizes the square of the errors of the discriminator classification of real and fake images. The discriminator $\mathcal{L}_{adv}^D$ and generator $\mathcal{L}_{adv}^G$ adversarial losses are:

$$\mathcal{L}_{adv}^D = \mathbb{E}_{x_t} [(D_{adv}(x_t) - 1)^2] + \mathbb{E}_{\tilde{x}_t} [(D_{adv}(\tilde{x}_t))^2] + \mathbb{E}_{\tilde{x}_{s|t}} \left[\sum_{s \in S} (D_{adv}(\tilde{x}_{s|t}))^2 \right]$$

$$\mathcal{L}_{adv}^G = \mathbb{E}_{\tilde{x}_t} [(D_{adv}(\tilde{x}_t) - 1)^2] + \mathbb{E}_{\tilde{x}_{s|t}} \left[\sum_{s \in S} (D_{adv}(\tilde{x}_{s|t}) - 1)^2 \right]$$

The domain classification objective leads the generator to synthesize images that are classified as having the intended target domain. For the generator, $\mathcal{L}_{dmn}^{fake}$ considers only generated images, whereas for the discriminator, $\mathcal{L}_{dmn}^{real}$ uses only real images. As a classification, they are calculated using cross entropy, given as:

$$\mathcal{L}_{dmn}^{fake} = \mathbb{E}_{\tilde{x}_t} [-\log(D_{dmn}(\tilde{x}_t))] + \mathbb{E}_{\tilde{x}_{s|t}} \left[\sum_{s \in S} D_{dmn}(\tilde{x}_{s|t}) \right]$$

$$\mathcal{L}_{dmn}^{real} = \mathbb{E}_{x_t} [-\log(D_{dmn}(x_t))]$$

To summarize, the full objectives of the generator $\mathcal{L}_G$ and the discriminator $\mathcal{L}_D$ are sums weighted by λ scalars given as:

$$\mathcal{L}_G = \mathcal{L}_{adv}^G + \lambda_{reg} \mathcal{L}_{reg} + \lambda_{mcy} \mathcal{L}_{mcy} + \lambda_{ssim} \mathcal{L}_{ssim} + \lambda_{dmn} \mathcal{L}_{dmn}^{fake}$$

$$\mathcal{L}_D = \mathcal{L}_{adv}^D + \lambda_{dmn} \mathcal{L}_{dmn}^{real}$$

In this extended version, we have added a new loss term to the generator called palette coverage loss, which is explained in Section 6 (Differentiable Palette Quantization).

3.2. Generator

The generator consists of four encoder branches, each processing a source image from a specific domain along with a channel-wise and

spatially distributed one-hot encoded label of the target domain. Each branch includes four downsampling blocks, followed by a bottleneck layer that concatenates the activation maps from all encoder branches and further refines them. The combined features are then fed into a shared decoder with four upsampling blocks. Skip connections link the concatenated encoder activations to their corresponding upsampling layers, preserving spatial details. Compared to the original architecture, our generator contains four encoder branches, while theirs has eight. Also, in this extended paper, our model receives a batch of color palettes as an additional input, which is used to constrain the generated colors to the desired palettes.

Each downsampling block consists of two sets of convolution (kernel size 3 with stride 1), instance normalization, and ReLU activation, followed by another convolution that reduces the spatial dimensions in half (kernel size and stride 2). In turn, an upsampling block consists of a concatenation of the activation maps with a skip connection, two sets of convolution (with a kernel size of 3 and a stride of 1), instance normalization, ReLU activation, followed by a transpose convolution that doubles the spatial dimensions (with a kernel size of 2 and a stride of 2).

The image size and number of channels we use is $64 \times 64 \times 4$, contrasted to the $128 \times 128 \times 3$ configuration of the original architecture. We increased the number of channels for each layer to improve the network capacity: they are four times the original, becoming 64, 128, 256, 512, and 1024 for the blocks in each encoder branch, and 1024, 512, 256, 128, and 64 for the decoder blocks.

3.3. Discriminator

The discriminator receives a batch of images and outputs values D_{adv} that should be one for real images and zero for the generated ones. In addition, it classifies the domain of each image, yielding probabilities D_{dmn} of them having each domain.

The network topology is the same as in the original CollaGAN, with 6 downsampling blocks, each consisting of a convolution that halves the resolution (with a kernel size of 4, stride 2) while increasing the number of channels, with a Leaky ReLU activation. The last block also contains a dropout layer (with 50% probability). Following it, two parallel convolutions represent the D_{adv} (1 filter with kernel size 3 and stride 1) and the D_{dmn} outputs (4 filters, kernel size 1, stride 1), with



Fig. 2. LEFT. Comparison of input dropout strategies: (a) no dropout, which always presents $|N| - 1$ images to the generator, (b) the original strategy [13] that randomly selects a number of additional images to drop out (equal chance), and (c) our conservative strategy that is also random but has higher chances of keeping more images as input. RIGHT. Resulting images from different replacement procedures during the training backward pass: (a) replaces the forward target image and all that have been dropped out by the image generated in the forward pass (original CollaGAN [13]) and (b) replaces only the forward target image by the one just generated (our proposal).

linear and softmax activations, respectively. The number of filters for the downsampling blocks are 64, 128, 256, 512, 1024, and 2048.

3.4. Training procedure

We select a batch of paired images x_S at each training step with random target domains t (they can be different for each example in the batch). To add more variability to the dataset, each example can undergo a data augmentation operation of rotating its hue by up to 180 degrees in the negative or positive direction with an 80% probability. The generator receives the batch of $\langle x_S, t \rangle$ and creates the missing $\hat{x}_t$, in the forward pass. Next, $\hat{x}_t$ is used in place of x_t to create a number of new batches equal to $|N| - 1$, in which each domain in S becomes the target, in the backward (or cyclical) pass. The generator then creates $\tilde{x}_{s|t}$, $\forall s \in S$ that must be as close as possible to the original x_s , $\forall s \in S$.

The CollaGAN architecture authors observed that images are much worse as the number of available sources decreases. However, it is common to have use cases in which more than one domain is missing. Hence, they proposed a batch selection strategy called input dropout, in which the model trains with one or more missing domains. For instance, for $|N| = 4$ and $t = d$, when a batch $\langle x_S, t \rangle$ is selected using the input dropout strategy, x_S can have zero, one or two withdrawn images and be one of the following:

$$\begin{aligned} x_S &= \{x_a, x_b, x_c\} & x_S &= \{x_{zero}, x_b, x_c\} & x_S &= \{x_{zero}, x_{zero}, x_c\} \\ & & x_S &= \{x_a, x_{zero}, x_c\} & x_S &= \{x_{zero}, x_b, x_{zero}\} \\ & & x_S &= \{x_a, x_b, x_{zero}\} & x_S &= \{x_a, x_{zero}, x_{zero}\} \end{aligned}$$

In the original CollaGAN, the number of images to be dropped out is chosen uniformly, leaving a 33% chance of having the full source domain set. That strategy improved the results in our task too. However, we observed that a more conservative approach in which the model trains more frequently dropping out few images yields even better results than fewer available images. We adopted chances of 10%, 30%, and 60% to have two, one, and zero images dropped out. Fig. 2 (left) compares the three strategies (no dropout, original dropout, conservative dropout) with different numbers of missing images.

Another change we made to the training procedure relates to the backward generation pass. When the cycled images $\tilde{x}_{s|t}$, $\forall s \in S$ (e.g., $\tilde{x}_{a|d}$, $\tilde{x}_{b|d}$, and $\tilde{x}_{c|d}$) are generated in the original implementation, the image $\hat{x}_t$ produced in the forward pass replaces not only the original target image x_t , but also all images that have been dropped out due to the batch selection strategy. We experimented with having $\hat{x}_t$ replace only x_t and observed better results.

To illustrate the difference, considering a batch with $t = d$ and the domain c dropped out, the backward generated images $\tilde{x}_{s|t}$, $\forall s \in S$ for

the original (left) and our implementation (right) would be as shown next. We highlighted the differences in bold:

$$\begin{aligned} \tilde{x}_{a|d} &= G(\{x_{zero}, x_b, \hat{x}_d, \hat{x}_d\}, a) & \tilde{x}_{a|d} &= G(\{x_{zero}, x_b, \mathbf{x}_{zero}, \hat{x}_d\}, a) \\ \tilde{x}_{b|d} &= G(\{x_a, x_{zero}, \hat{x}_d, \hat{x}_d\}, b) & \tilde{x}_{b|d} &= G(\{x_a, x_{zero}, \mathbf{x}_{zero}, \hat{x}_d\}, b) \\ \tilde{x}_{c|d} &= G(\{x_a, x_b, x_{zero}, \hat{x}_d\}, c) & \tilde{x}_{c|d} &= G(\{x_a, x_b, x_{zero}, \hat{x}_d\}, c) \end{aligned}$$

Fig. 2 (right) compares the generated images when the model trains using the original replacement procedure for the dropped-out images versus our version where only the forward target image is replaced by the one generated in the forward pass. We can note that with the original procedure, the generator produces images with artifacts from domains other than the target one, mostly noticeable through the wrong number of eyes in the examples.

Regarding the number of trainable parameters, the generator contains 104,887,616 values, and the discriminator has 44,726,272. After training, the generator takes ≈ 110 ms to produce an image using a GeForce GTX 1050 Ti GPU.

4. Methodology

This section outlines how we evaluated our model in the task of generating missing poses while receiving a different number of input images. We first introduce the datasets employed in our study, followed by a description of the evaluation metrics used to assess the quality of the generated images — specifically, the L_1 distance to the ground truth and the Fréchet Inception Distance (FID). We conclude by presenting the baseline models included in our comparative analysis.

4.1. Dataset

We assembled pixel art character sprites from multiple sources, including scraped assets from free game resource websites (e.g., opengameart.org), and also put together new characters by combining different body parts. We named this collection the Pixel Art Characters (PAC) Dataset and have made it publicly available.²

The PAC Dataset contains 14,202 paired images of character sprites facing four directions: back, left, front, and right, representing diverse art styles. The collection primarily features humanoid characters of varying sizes, along with a smaller number of animal, vehicle, and monster sprites. As illustrated in Fig. 3, the dataset exhibits significant visual variety. Since source images had inconsistent dimensions, we standardized them by padding smaller sprites with transparency to

² PAC Dataset: <https://huggingface.co/datasets/pluckshire/pac>.



Fig. 3. Sample images from the dataset showing different sizes/art styles (columns) facing four directions (rows).

match the largest size (64×64 pixels). We generated an alpha channel for images lacking transparency data based on character silhouettes, resulting in consistent RGBA formatting across all entries. The dataset is split into 12,074 training examples (85%) and 2,128 test examples.

4.2. Evaluation metrics

The evaluation of generative models remains an active area of research, with numerous metrics proposed in recent literature [29]. For quantitative evaluation, we employ two established metrics:

1. **L_1 distance** (mean absolute error), which measures pixel-wise color differences between generated and target images.
2. **Fréchet Inception Distance (FID)** [14], which compares feature distributions of the two image sets using Inception v3 [30].

Both metrics are distance-based, meaning lower values indicate better alignment with the target (with zero denoting perfect similarity). However, since these metrics do not always correlate with human judgment, we also conduct visual inspections to assess perceptual fidelity.

4.3. Baseline models

We compare our model with two other architectures proposed for the image-to-image translation task: Pix2Pix [31] and StarGAN [32].

Pix2Pix. We trained a modified version of the architecture proposed in Coutinho and Chaimowicz [8] for generating pixel art characters in a target pose given an image of it in a source one. Differently from the referenced work, we use 12 such models to support translation from and to all four poses: back, left, right, and front, excluding models from and to the same direction. Each generator has 29,307,844 trainable variables, so the model collection contains 351,694,128 parameters.

StarGAN. We trained a StarGAN-based model to perform multi-domain translation using a single generator and discriminator pair. The original generator receives the source image and a label indicating the target direction. Still, we found that providing a label of the *source* domain increases the quality of the generated characters. In turn, the original critic receives only the image to be evaluated. However, we got better results by sending the source image too (before translation), which makes it perform a *conditional* discrimination. In that case, the network indicates whether the provided image is real/fake *considering that* it is a translation of the source image. For a fair comparison, we train the model using supervision (the original trains without paired images). The generator contains 134,448,128 parameters.

Table 1

FID and L_1 distance to the ground truth of the images generated by the baselines Pix2Pix and StarGAN receiving a single domain as input and our model receiving three (Ours-3).

Target	Average FID			Average L_1		
	Pix2Pix	StarGAN	Ours-3	Pix2Pix	StarGAN	Ours-3
Back	5.788	3.378	2.054	0.05402	0.06429	0.04530
Left	2.380	1.250	1.037	0.04934	0.06344	0.03439
Front	5.392	3.156	1.955	0.05875	0.07263	0.04985
Right	2.806	1.368	0.987	0.04880	0.06273	0.03360
Average	4.091	2.288	1.508	0.05273	0.06577	0.04078

5. Experiments

The model trained with the PAC dataset for 240,000 generator update steps in minibatches of 4 examples, equivalent to ≈ 80 epochs. It took 22 hours to train using a GeForce RTX 4090 GPU. We used early stopping to select the model that had the best metrics on its test set instead of getting the one in the end to prevent overfitting. We evaluate the model at every 1,000 update steps and select the one with the lowest (best) L_1 value throughout the training procedure. We applied hue rotation to each character with 80% probability as data augmentation.

The generator and discriminator optimize their weights using Adam, with $\beta_1 = 0.5$ and $\beta_2 = 0.999$, and a learning rate that starts as 0.0001 and linearly decays to zero during the second half of the training. The parameters of the objective function were $\lambda_{reg} = 100$, $\lambda_{dmn} = 10$, $\lambda_{ssim} = 10$, and $\lambda_{mcy} = 10$.

In the following experiments, we start by evaluating the model's performance using three input images (dubbed Ours-3) against the baselines. Next, we evaluate the same model (trained with three source domains) in the scenario of it receiving only two (Ours-2) and one (Ours-1) input images. We then follow up with an experiment to assess different input dropout strategies and an ablation study of the changes proposed atop the original CollaGAN.

5.1. Missing image imputation

We trained our proposed model using conservative input dropout and the forward-only replacer strategy. Table 1 shows the values of FID and L_1 distance for our proposed model and the baselines, with the rows representing the target pose and the columns displaying the metrics for the baselines Pix2Pix and StarGAN, averaged considering the translation from the other source domains and our model with the three other domains as input.

Regarding FID, Ours-3 had the lowest (best) values of the evaluated models in all target poses and, hence, on average, too: 1.508 (Ours-3) versus 2.288 (StarGAN) and 4.091 (Pix2Pix). Also, the L_1 distance for our model was the lowest (best), with the averages 0.04078 (Ours-3), 0.05273 (Pix2Pix), and 0.06577 (StarGAN). As we expected, our model utilizes the information from the additional poses to enhance the quality of the generated images. We demonstrate this in the next experiment, showing how it performs when given fewer input images. Next, we visually analyze the generated images.

Fig. 4 shows examples of generated images using the different models, with each row having a different domain as the target. The columns for Pix2Pix and StarGAN show three images per row. As they are models that take a single image as input, we depict the image generated for the target pose from each other domain. In contrast, our model uses all the other domains as input and has a single generated image for each row.

The quality of the generated images varies with the model and the target pose. We analyze the results qualitatively according to the use of colors and the generated shape. Regarding the former, all models generate images with colors in meaningful positions but employ many

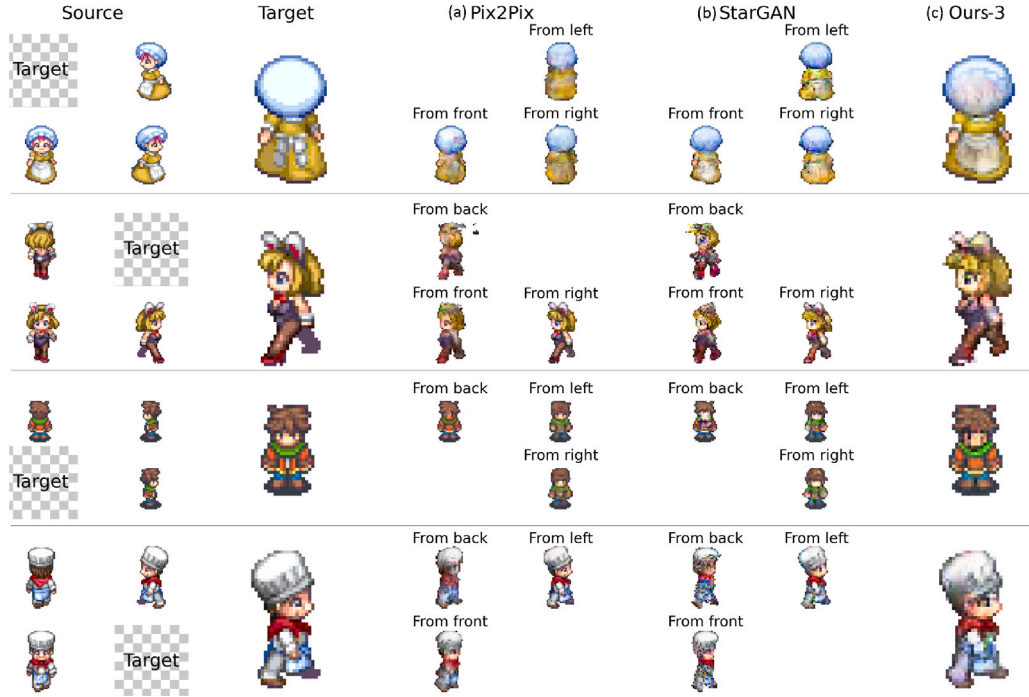


Fig. 4. Example images generated in different target domains. The **columns** show the source images, the target, and the generated images using a single source domain by the (a) Pix2Pix baseline and the (b) StarGAN baseline, and using all available source images with (c) our proposed model (Ours-3).

variations of the same tones instead of restricting to a small palette. In Section 6 (Differentiable Palette Quantization), we propose a method that trains our model end-to-end to produce images that strictly adhere to a desired color palette.

Regarding the shape, the poses imputed by our model are very close to the intended, and so are the ones generated by the baseline models that translate images from left to right and vice-versa. Generating images in that scenario usually consists of learning a horizontal flip transformation, which is an easier task endorsed by the lower FID and L_1 values when the target is left or right. When the target is back, a noticeable artifact is the faint presence of details from the character's face, especially prominent in the images generated with our model.

Visual inspection of the results shows that the quality of the images generated by our model is either on par or better than the baselines. We highlight that our architecture contains 104,887,616 trainable parameters, which is 22% smaller than StarGAN and 70% than the collective Pix2Pix. Next, we assess how the model performs when fewer images are available.

5.2. Generating from fewer domains

Even though we propose a model to impute a single missing domain, we also evaluate it in scenarios where it receives two (Ours-2) or only one image (Ours-1).

Table 2 shows the FID and L_1 distance between the generated images and the ground truth, with different source domain combinations on the rows and the target domains in the columns. The first row grouping shows the metrics for the model evaluated with a single input image, the second with two images, and the third with three. Each section ends with a row with the average values for all source combinations for each target domain.

Analyzing Table 2 by sweeping the columns downwards (following the increase in the number of inputs), we observe that both metrics' average values consistently reduce (i.e., improve) as we introduce more inputs to the generator. For instance, taking the FID value averaged over the source domain combinations for the right target pose, the values are 4.803 (Ours-1), 2.769 (Ours-2), and 0.987 (Ours-3).

Similarly, the best source combinations at each number of input grouping (values highlighted in the table) improve as the generator receives more inputs in most cases, except in two situations that we describe later. To illustrate, looking at the FID values when the target is the back pose, the best source directions are front (2.707, for Ours-1), left+front (2.236, for Ours-2), and left+front+right (2.054, for Ours-3). In addition, the sources that yielded the highest (worst) values for each target pose also improved as the number of inputs increased. In the same example of generating the back pose, the worst sources are right (19.117, Ours-1), left+right (13.574, Ours-2), and left+front+right (2.054, Ours-3).

Fig. 5 portrays example characters generated by our model with each possible source domain combination. We can see how the generated images get closer to the ground truth as the number of input domains increases. At the same time, some examples have promising images even with a single input domain, especially for the easier left/right and front/back translation directions.

The two exceptions to the monotonically decreasing metric values are the L_1 distance between the best values for Ours-2 and Ours-3 when generating left and right poses. When going from two to three inputs, the L_1 distance increases (worsens) when the target is left (0.03334 for f+r to 0.03439 for b+f+r) and right (0.03284 for l+f to 0.03360 for b+l+f).

We can visually analyze the generated images to investigate the divergence between FID and L_1 in this situation. Fig. 6 depicts left-facing characters created from the right+back, right+front, and right+back+front source domains. Although the differences are subtle, the images generated from the three inputs are crispier and present more fine-grained details (e.g., the better separation of hair locks). The lower L_1 values for the images generated from only two poses can be attributed to the more blurry pixels, as that metric evaluates the pixel-wise average color distance and does not account well for the crispy nature of pixel art.

5.3. Input dropout

We evaluated different batch selection strategies to determine their effect on the model performance. Specifically, we investigated whether

Table 2

FID and L_1 distance for all input images (rows, with each pose represented by its initial letter) and target domain (columns) combinations for our proposed model. The numbers in **bold** indicate the best combination of source domains for each target pose.

	Target Source	FID				L_1			
		Back	Left	Front	Right	Back	Left	Front	Right
Ours-1	b	–	8.581	2.735	7.085	–	0.07463	0.05559	0.07835
	l	16.401	–	13.345	1.764	0.07254	–	0.07456	0.03480
	f	2.707	5.168	–	5.562	0.04971	0.06940	–	0.07217
	r	19.117	2.157	16.093	–	0.07630	0.03516	0.08066	–
	Average	12.741	5.302	10.725	4.803	0.06619	0.05973	0.07027	0.06177
Ours-2	b+l	–	–	2.463	1.771	–	–	0.04996	0.03342
	b+f	–	4.909	–	4.977	–	0.06728	–	0.07140
	b+r	–	1.730	2.620	–	–	0.03395	0.05028	–
	l+f	2.236	–	–	1.559	0.04637	–	–	0.03284
	l+r	13.574	–	11.155	–	0.06894	–	0.07033	–
	f+r	2.602	1.725	–	–	0.04607	0.03334	–	–
	Average	6.137	2.788	5.413	2.769	0.05380	0.04485	0.05686	0.04589
Ours-3	b+l+f	–	–	–	0.987	–	–	–	0.03360
	b+l+r	–	–	1.955	–	–	–	0.04985	–
	b+f+r	–	1.037	–	–	–	0.03439	–	–
	l+f+r	2.054	–	–	–	0.04530	–	–	–
	Average	2.054	1.037	1.955	0.987	0.04530	0.03439	0.04985	0.03360

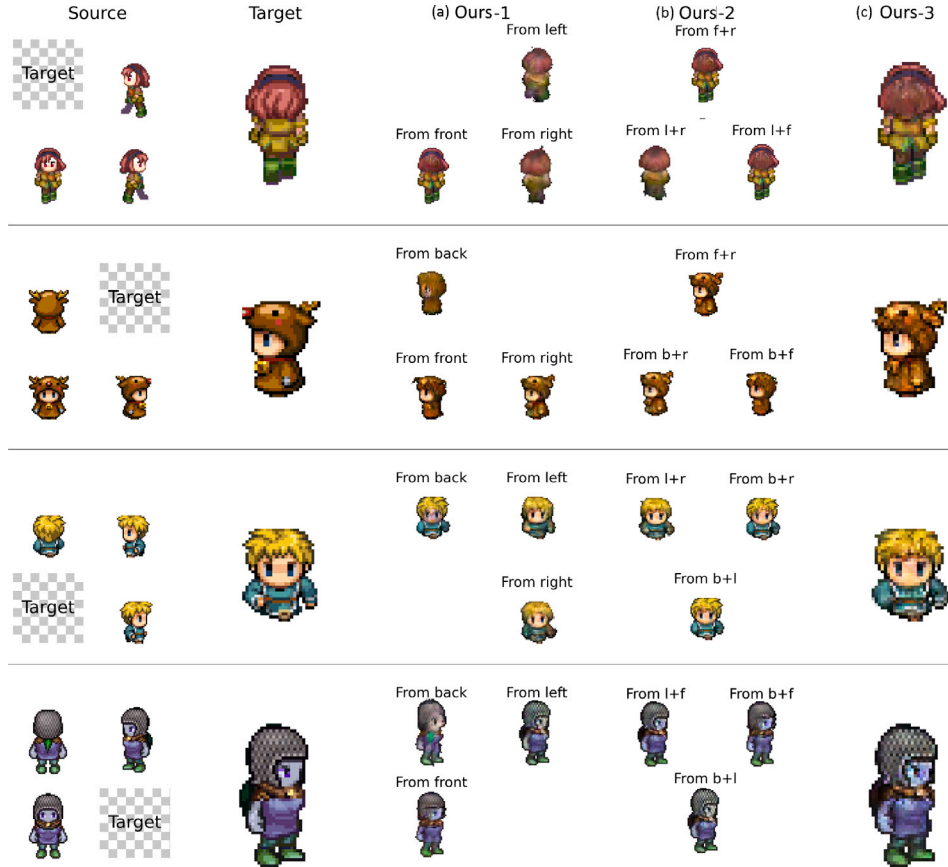


Fig. 5. Images generated by our proposed model with different source domain combinations. **Columns** show the source and target images, and the ones generated (a) with a single input image (Ours-1), (b) two images (Ours-2) and (c) all three (Ours-3). **Rows** show examples with different target poses.

consistently presenting all three available domains during training yields superior results compared to intermittently omitting certain domains.

We assessed always showing all available domains (none), the original input dropout strategy proposed in Lee et al. [13], a curriculum learning approach suggested by Sharma and Hamarneh [33], and our proposed conservative tactic.

The original approach has an equal chance of presenting three, two, or a single image in a training step. The curriculum learning approach starts training with easier tasks (using three images) and progressively makes it harder (using a single input) until half of the training, then it randomly chooses between the number of domains to drop out for the second part. Lastly, the conservative approach randomly selects the number of images to drop, but with higher probabilities to keep

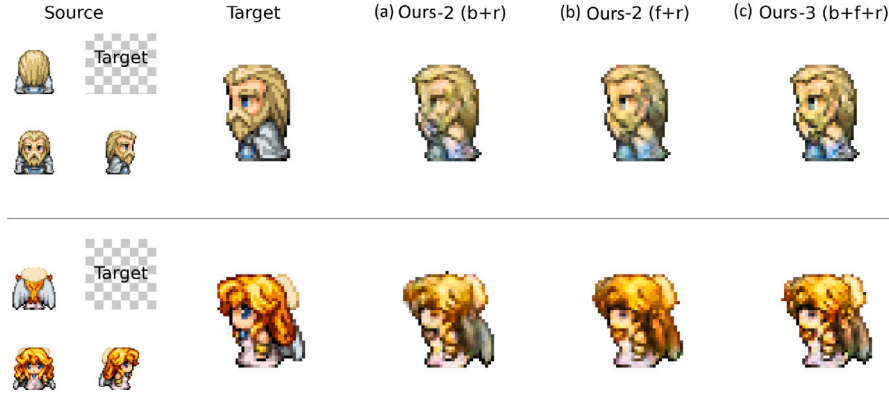


Fig. 6. Comparison of our model using (a, b) two images (Ours-2 (b+r) and Ours-2 (f+r)) and (c) three images as input (Ours-3). When the target is either the left or right poses, introducing a third source input (i.e., from Ours-2 to Ours-3) results in crispier, more detailed images.

Table 3

FID and L_1 distance to the ground truth of different input dropout strategies when the model receives 3, 2 or 1 images as input. The columns show (a) No input dropout, (b) the original strategy [13], (c) a curriculum learning approach [33], and (d) our conservative strategy.

Sources		(a) None	(b) Original	(c) Curriculum	(d) Conservative
FID	Ours-3	4.816	1.911	2.160	1.508
	Ours-2	19.050	6.835	9.233	4.277
	Ours-1	32.676	11.162	20.303	8.393
	Average	18.847	6.636	10.566	4.726
L_1	Ours-3	0.04523	0.04277	0.04222	0.04078
	Ours-2	0.08003	0.05053	0.07389	0.05035
	Ours-1	0.12820	0.06243	0.12232	0.06449
	Average	0.08449	0.05191	0.07948	0.05187

more images: 60% with 3 images, 30% with 2, and 10% with a single image.

Table 3 presents the results from the models trained with the different input dropout strategies (columns) in the scenarios of having three, two, or one available image as input (rows). We can observe that using any input dropout yields better results than always showing all domains (none).

Compared to the original and curriculum learning strategies, our proposed conservative tactic has better FID and L_1 metrics on the average of the three scenarios. In particular, regarding FID, the model trained with the conservative input dropout worsens its performance less drastically with the decrease of input domains. Regarding L_1 , its metrics are better than the other models when two and three images are available.

5.4. Ablation study

To understand the impact of our changes to the original CollaGAN architecture, we trained and evaluated models that progressively added each modification. Table 4 shows the FID and L_1 values of the generated images averaged over all domains and among the scenarios of the model receiving three, two, and one input domains. The rows show the results of each modification cumulatively: the first one is the original CollaGAN model without any of our proposed changes, the second introduces the first modification, the third uses two changes, and the last includes all three (our final model).

The original model had 6,565,712 trainable variables, but with the increased capacity, there are 104,887,616 parameters. That change alone improved L_1 but worsened FID. The replacement strategy of substituting only the original target with the image generated in the forward step improves both metrics' results. Lastly, training with the proposed conservative input dropout further enhances the results, with FID and L_1 values that are 46.7% and 14.53% better than the original architecture.

Table 4

Performance of the modifications made to the original CollaGAN architecture.

Modification(cumulative)	Average FID		Average L_1	
	Value	Improv.	Value	Improv.
Original	8.866	–	0.06069	–
+ Increased capacity	11.078	–24.95%	0.05666	6.64%
+ Forward Replacer	6.636	25.15%	0.05191	14.47%
+ Conservative Inp. Drop.	4.726	46.70%	0.05187	14.53%

5.5. Discussion

We summarize the quantitative results in Table 5. It contains the FID and L_1 distance for a task with medium difficulty (translating front to right) and an average value among all source and target domain combinations for each model.

As the metrics do not converge, we analyze them separately. The FID of the models with a single generator and discriminator pair (i.e., StarGAN and our approach) are lower (hence, better) than the cross-domain model (i.e., Pix2Pix). Regarding the L_1 distance, which strongly penalizes shape mismatch, Pix2Pix has a better value than StarGAN and our proposed model with a single input but is worse than our model with two and three source domains. Our model performs the best according to both metrics when the three other domains are available.

Quantitative results aside, a visual inspection of the generated images shows that the quality varies greatly, depending on the translation direction, the model, and the similarity of the input character to the style of those in the training set. While some of the generated images are readily usable, others are too distant from the ground truth to be useful.

In particular, when the input characters are very different from the art styles represented in the training set, the generated images can

Table 5

A summary of FID and L_1 averaged over all possible source and target domain combinations (except $t = s$ and $t \in S$) for the proposed models.

	Task	Pix2Pix	StarGAN	Ours-1	Ours-2 ^a	Ours-3
FID	Front→Right	3.368	1.565	5.562	4.977	0.987
	Average	4.091	2.288	8.393	4.277	1.508
L_1	Front→Right	0.05788	0.07061	0.07217	0.07140	0.03360
	Average	0.05273	0.06577	0.06449	0.05035	0.04078

^a The sources for Ours-2 in the Front→Right task on the table are Front + Back.

exhibit very low quality and not resemble pixel art characters. That indicates a limitation in the generalization capacity of the models. Furthermore, even the best generated images contain many small variations of the same colors. In the next section, we propose a method to overcome that problem.

6. Differentiable palette quantization

The images generated by our proposed model and those produced by the baseline exhibit a significantly broader color distribution than expected for pixel art, which typically adheres to a strict color palette comprising a limited number of colors (e.g., 15 to 50).

In Section 2.2 (Pixel Art Generation), we reviewed existing approaches to enhance the quality of generated images by incorporating art style-specific constraints. Several methods focus on guiding or restricting color usage to align with a predefined palette. This requirement is particularly critical for designers who may employ generative models to streamline asset creation, as deviations from the palette would necessitate extensive manual recoloring—a time-consuming and repetitive task.

One common solution involves post-processing the generated images by quantizing each pixel's color to the nearest match in a target palette [8]. While effective in improving visual quality and usability, this approach is inherently non-differentiable, requiring an additional step in the workflow.

The primary challenge in integrating palette quantization in a model training end-to-end lies in the non-differentiability of operations such as finding the palette color with the minimum distance to the pixel (`tf.argmax`) or assigning an untracked value to an output (i.e., replacing the color of a pixel), which are incompatible with gradient-based optimization. To address this, Binnering and Sorkine-Hornung [26] proposed leveraging the Gumbel-Softmax reparameterization trick, enabling differentiable sampling from a discrete color palette.

Our work introduces a related but distinct approach to enforce palette adherence by employing a softmax with temperature function and a weighted-average color replacement method. Both approximate non-differentiable operations while preserving gradient flow. Furthermore, we propose an auxiliary palette coverage loss to encourage the generator to utilize the full diversity of the target palette, mitigating color underutilization.

6.1. Softmin with temperature

The seminal work of Kirkpatrick et al. [34] introduced simulated annealing, a stochastic optimization method inspired by thermodynamic cooling processes, which balances exploration and exploitation through a temperature-controlled acceptance criterion. This concept was later adapted to modern deep learning [35] in the context of knowledge distillation, where a temperature-scaled softmax enables smoother probability distributions for transferring knowledge from a teacher to a student network.

Our work employs a softmax variant to enable differentiable color quantization. While the `argmin` operation performs a hard selection from a categorical distribution, the *softmax with temperature* provides

a probabilistic alternative, allowing gradient-based optimization over discrete decisions (i.e., selecting a palette color). Given a vector of distances $\mathbf{d}$, the softmax is defined as:

$$\text{softmax}(\mathbf{d}, \tau)_i = \frac{e^{-d_i/\tau}}{\sum_j e^{-d_j/\tau}}$$

where the temperature τ controls the sharpness of the distribution — high τ yields nearly uniform probabilities, while $\tau \rightarrow 0$ approaches one-hot argmin.

This approach allows gradients to flow through sampling operations, making it suitable for end-to-end training. As with simulated annealing [34], the temperature is initially high and slowly decays, approaching zero at the end of training.

6.2. Weighted-average replacement

After computing the softmax distances between each pixel and the palette colors, we replace each pixel's color with a weighted average of all palette colors, where the weights are derived from their distances in RGBA space.

Formally, let $I \in \mathbb{R}^{H \times W \times C}$ denote an image and $P \in \mathbb{R}^{K \times C}$ a target palette with K colors, where $H \times W$ is the image resolution and C the number of channels. The squared L_2 distance between pixel I_{hw} and palette color P_k is given by $D_{hw,k}$:

$$D_{hw,k} = \|I_{hw} - P_k\|_2^2$$

The quantized image $I_{hw}^{P\text{-quantized}}$ is then obtained via the weighted average:

$$I_{hw}^{P\text{-quantized}} = \sum_{k=1}^K \text{softmax}(D_{hw,k}, \tau)_k \cdot P_k$$

This approach ensures differentiability, as the weighted average operation admits well-defined gradients for backpropagation. The temperature τ is initialized to a moderate value (e.g., $\tau = 0.1$) and decays linearly to zero during training. Consequently, the generated images transition from smooth color interpolations early in training to strict palette adherence at convergence. Notably, the softmax guarantees that all output colors lie within the convex hull of the palette, preserving at least softly the artistic constraints throughout the optimization.

Fig. 7 illustrates the impact of the temperature parameter τ in the palette quantization with softmax temperature and weighted-average replacement. For higher τ values, the generator produces smoothly blended colors approaching the palette mean, whilst when $\tau \rightarrow 0$, the quantization forces exact palette adherence. When $\tau = 0$, the generated images employ a color palette closely approximating that of the target images, though not identical due to two key factors:

- Palette discrepancy:** we provide the generator with the desired palette extracted from the union of the known source images as input. This may result in a palette that is slightly different from the ground truth of the missing target image. For example, the target image might use a few colors not seen on any other pose used as input.
- Partial palette utilization:** while our palette quantization procedure ensures that generated images exclusively employ colors from the target palette, it does not enforce the use of all palette colors, leading to potential color underutilization.

Next, we present our strategy to address this issue of color underuse.

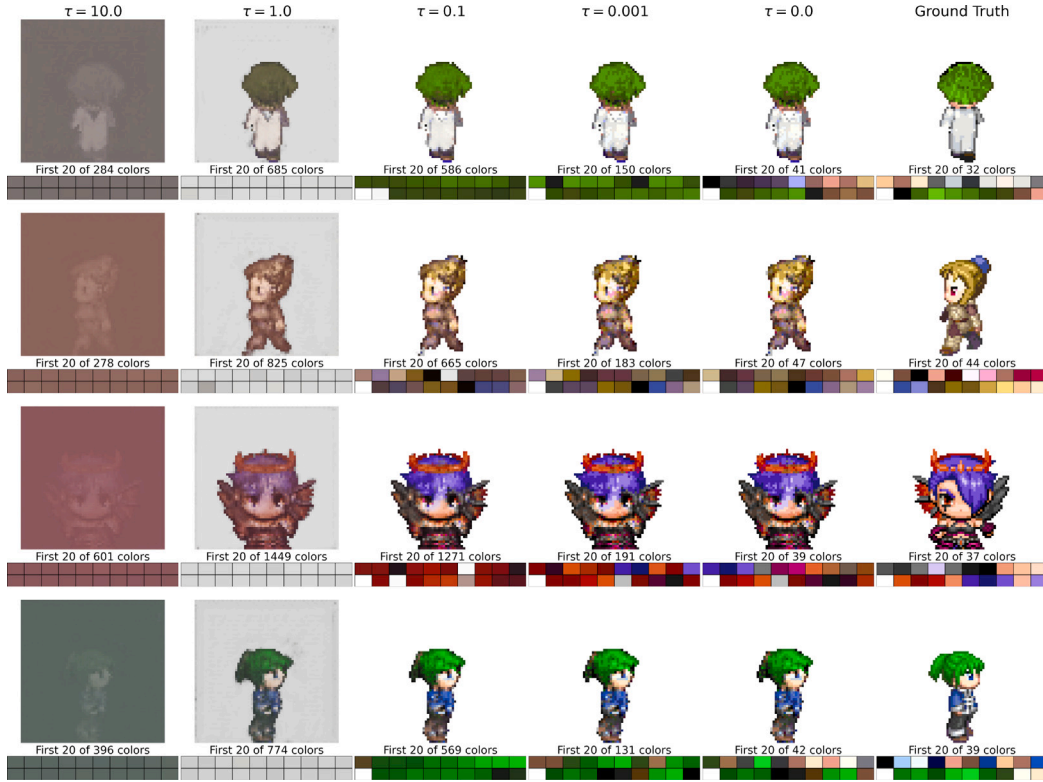


Fig. 7. Visualization of temperature parameter (τ) effects on generated pixel art using differentiable palette quantization. High τ values produce smoothly blended colors approaching the palette mean, demonstrating the softmin operation's probabilistic nature during early training stages. As $\tau \rightarrow 0$, the quantization becomes deterministic, forcing exact palette adherence while maintaining gradient flow. The progression illustrates our method's annealing strategy, where τ is gradually reduced during training to achieve color diversity and strict palette compliance.

6.3. Palette coverage loss

While the palette quantization ensures that generated images adhere strictly to the prescribed color palette, this constraint alone does not guarantee that all palette colors are actively utilized. For instance, the model could theoretically produce valid outputs using only a single color from the palette while entirely neglecting others.

Empirically, we observed this limitation in cases where fine details – such as a character's eyes, which often occupy only a few pixels – were rendered without their intended palette colors. To address this, we introduce a *palette coverage loss* ($\mathcal{L}_{pcov}$) that explicitly encourages the generator to employ the full diversity of the palette.

We break down the loss calculation in three steps:

Soft Color Assignment: For each pixel I_{hw} , we compute its soft assignment probability $\alpha_{hw,k}$ to palette color P_k using the temperature-scaled softmin:

$$\alpha_{hw,k} = \frac{e^{-D_{hw,k}/\tau}}{\sum_{j=1}^K e^{-D_{hw,j}/\tau}}.$$

Color Usage Estimation: The aggregate usage β_k of each palette color P_k is obtained by summing its soft assignments across all pixels:

$$\beta_k = \sum_{h=1}^H \sum_{w=1}^W \alpha_{hw,k}.$$

Underutilization Penalty: The coverage loss penalizes underused colors ($\beta_k < 1$) via a hinge function, ensuring gradients only propagate for neglected colors. The final loss averages $\mathcal{L}_{pcov,k}$ over all K palette colors:

$$\mathcal{L}_{pcov,k} = \max(1 - \beta_k, 0)$$

$$\mathcal{L}_{pcov} = \frac{1}{K} \sum_{k=1}^K \mathcal{L}_{pcov,k}.$$

The $\mathcal{L}_{pcov}$ objective is added to the generator loss function $\mathcal{L}_G$ scaled by λ_{pcov} , which we empirically selected as 1. Hence, the final loss equation for the generator becomes:

$$\mathcal{L}_G = \mathcal{L}_{adv}^G + \lambda_{reg} \mathcal{L}_{reg} + \lambda_{mcy} \mathcal{L}_{mcy} + \lambda_{ssim} \mathcal{L}_{ssim} + \lambda_{dnn} \mathcal{L}_{dnn}^{fake} + \lambda_{pcov} \mathcal{L}_{pcov}$$

Fig. 8 compares character generations under different quantization strategies: (1) no palette quantization, (2) post-process quantization (as in Coutinho and Chaimowicz [8]), and our end-to-end quantization approach, (3) without and (4) with the palette coverage loss.

Analysis of the generated images reveals that palette quantization significantly enhances output quality, particularly in terms of utility, by reducing the number of colors—a critical feature for pixel art. However, post-process quantization (4th column) frequently introduces incorrect color assignments. In contrast, our end-to-end quantization approach (5th column) produces results more consistent with the ground truth, though it may still underutilize certain palette colors. Including the palette coverage loss (6th column) further mitigates this under-utilization, as evidenced by more accurate eye color representation.

7. Final remarks

In this work, we formulated pixel art character generation as a missing data imputation problem and addressed it using a deep generative model based on the CollaGAN architecture. Our key contributions include: (1) a generative model that can use all the available poses of character to impute a missing one; (2) architectural modifications to the original CollaGAN that improve the quality of the generated images; and (3) a novel differentiable color palette quantization method



Fig. 8. Comparison of images generated with our proposed model using different palette quantization mechanisms. The **rows** show examples with different target domains, and the **columns** display the input and target images, the image (a) from Ours-3 (no quantization), (b) from our model but with quantization as a post-process step [8], (c) our new embedded quantization approach (end-to-end training), and (d) the embedded quantization with the additional palette coverage loss (our final model).

that enables end-to-end training while enforcing strict adherence to a desired color palette.

Compared to baseline approaches, our method generates comparable or superior-quality images when three input domains are available. Performance degrades gracefully in scenarios with fewer input images, though with reduced output fidelity—a trade-off inherent to the data imputation setting.

For future work, we identify a few promising research directions: (a) disentangled representations to explore content-style separation methods [36] to handle domain-invariant features better; (b) different forms to fuse the features from each domain, as proposed in [37,38]; and (c) diffusion models by adapting latent diffusion methods [39] to leverage their inherent multi-modality for diverse output generation.

CRediT authorship contribution statement

Flávio Coutinho: Writing – original draft, Software, Investigation, Visualization, Methodology, Conceptualization. **Lucas G.S. Chaves:** Software, Writing – review & editing, Investigation. **Luiz Chaimowicz:** Writing – review & editing, Resources, Funding acquisition, Supervision, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially supported by CAPES, Brazil, CNPq, Brazil and Fapemig, Brazil.

Data availability

The PAC dataset (<https://huggingface.co/datasets/plucksquire/pac>) we used is and our reference source code (<https://github.com/fegemo/multi-domain>) are available online.

References

- [1] J. Schreier, Blood, sweat, and pixels: The triumphant, in: *Turbulent Stories behind how Video Games are Made*, HarperCollins, New York, 2017.
- [2] D. Silber, *Pixel Art for Game Developers*, CRC Press, Boca Raton, 2015.
- [3] R.D. Moreira, F. Coutinho, L. Chaimowicz, Analysis and compilation of normal map generation techniques for pixel art, in: 2022 21st Brazilian Symposium on Computer Games and Digital Entertainment (SBGames), IEEE, Natal, 2022, pp. 1–6, <http://dx.doi.org/10.1109/SBGAMES56371.2022.9961116>.
- [4] F. Coutinho, L. Chaimowicz, Generating pixel art character sprites using GANs, in: 2022 21st Brazilian Symposium on Computer Games and Digital Entertainment (SBGames), IEEE, Natal, Brazil, 2022, pp. 1–6, <http://dx.doi.org/10.1109/SBGAMES56371.2022.9961120>.
- [5] D. Loftsdottir, M. Guzdial, SketchBetween: Video-to-video synthesis for sprite animation via sketches, in: Proceedings of the 17th International Conference on the Foundations of Digital Games, ACM, New York, 2022, pp. 1–7, <http://dx.doi.org/10.1145/3555858.3555928>.
- [6] A. Saravanan, M. Guzdial, Pixel VQ-VAEs for improved pixel art representation, in: Experimental AI in Games Workshop (EXAG) 2022, Pomona, 2022, pp. 1–9, <http://dx.doi.org/10.48550/arxiv.2203.12130>, ArXiv.
- [7] S. Hong, S. Kim, S. Kang, Game sprite generator using a multi discriminator GAN, KSII Trans. Internet Inf. Syst. 13 (2019) 4255–4269, <http://dx.doi.org/10.3837/tis.2019.08.025>.
- [8] F. Coutinho, L. Chaimowicz, Pixel art character generation as an image-to-image translation problem using GANs, Graph. Model. 132 (2024) 101213, <http://dx.doi.org/10.1016/j.gmod.2024.101213>.
- [9] Y.R. Serpa, M.A.F. Rodrigues, Towards machine-learning assisted asset generation for games: A study on pixel art sprite sheets, in: 2019 18th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames), IEEE, Rio de Janeiro, 2019, pp. 182–191, <http://dx.doi.org/10.1109/SBGAMES.2019.00032>, volume 2019-October.
- [10] J.-I. Choi, S.-K. Kim, S.-J. Kang, Image Translation Method for Game Character Sprite Drawing, Comput. Model. Eng. Sci. 131 (2022) 747–762, <http://dx.doi.org/10.32604/cmes.2022.018201>.
- [11] Y.R. Serpa, M.A.F. Rodrigues, Human and machine collaboration for painting game assets with deep learning, Entertain. Comput. 43 (2022) 100497, <http://dx.doi.org/10.1016/j.entcom.2022.100497>.
- [12] Flávio Coutinho, Luiz Chaimowicz, A Missing Data Imputation GAN for character sprite generation, in: Proceedings of the XXIII Brazilian Symposium on Games and Digital Entertainment (SBGames), 2024, p. 20, <http://dx.doi.org/10.5753/sbgames.2024.241116>.
- [13] D. Lee, J. Kim, W.-J. Moon, J.C. Ye, CollaGAN: Collaborative GAN for missing image data imputation, in: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, IEEE, Long Beach, 2019, pp. 2482–2491, <http://dx.doi.org/10.1109/CVPR.2019.00259>, volume 2019-June.
- [14] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, S. Hochreiter, GANs trained by a two time-scale update rule converge to a local Nash equilibrium, Adv. Neural Inf. Process. Syst. 2017 (2017) 6627–6638, <http://dx.doi.org/10.5555/3295222.3295408>.
- [15] I.A. Nikolov, Light the sprite: Pixel art dynamic light map generation, in: Annual Conference of the European Association for Computer Graphics, The Eurographics Association, London, United Kingdom, 2025, p. 4, <http://dx.doi.org/10.2312/EGS.20251032>.
- [16] O. Ronneberger, P. Fischer, T. Brox, U-net: Convolutional networks for biomedical image segmentation, in: Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 9351, 2015, pp. 234–241, http://dx.doi.org/10.1007/978-3-319-24574-4_28.
- [17] A. van den Oord, O. Vinyals, K. Kavukcuoglu, Neural discrete representation learning, Adv. Neural Inf. Process. Syst. 30 (2017) 1–10, <http://dx.doi.org/10.5555/3295222.3295378>, Curran Associates, Inc..
- [18] A. van den Oord, N. Kalchbrenner, K. Kavukcuoglu, Pixel recurrent neural networks, in: Proceedings of the 33rd International Conference on International Conference on Machine Learning, vol. 48, JMLR.org, New York, NY, USA, 2016, pp. 1747–1756, doi: 3045390.3045575.
- [19] P. Löwenström, Guiding Generation of 2D Pixel Art Characters using Text-Image Similarity Models: A Comparative Study of Generating 2D Pixel Art Characters using PixelDraw and Diffusion Model Guided By Text-Image Similarity Models, (Ph.D. thesis), KTH, School of Electrical Engineering and Computer Science (EECS), 2024.
- [20] M.A. Mercioni, R.B. Cazacu, S. Holban, Innovative pixel art: Generation and augmentation with pixray, in: 2024 16th International Symposium on Electronics and Telecommunications, ISETC 2024, Institute of Electrical and Electronics Engineers Inc., Timisoara, Romania, 2024, p. 4, <http://dx.doi.org/10.1109/ISETC63109.2024.10797335>.
- [21] P. Esser, R. Rombach, B. Ommer, Taming transformers for high-resolution image synthesis, Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (2020) 12868–12878, <http://dx.doi.org/10.1109/CVPR46437.2021.01268>.
- [22] F. Coutinho, L. Chaimowicz, On the Challenges of Generating Pixel Art Character Sprites Using GANs, Proc. AAAI Conf. Artif. Intell. Interact. Digit. Entertain. 18 (2022) 87–94, <http://dx.doi.org/10.1609/AAIDE.V18I1.21951>.
- [23] Z. Wu, L. Chai, N. Zhao, B. Deng, Y. Liu, Q. Wen, J. Wang, S. He, Make Your Own Sprites: Aliasing-Aware and Cell-Controllable Pixelization, ACM Trans. Graph. 41 (2022) 1–16, <http://dx.doi.org/10.1145/3550454.3555482>.
- [24] J.-Y. Zhu, T. Park, P. Isola, A.A. Efros, Unpaired image-to-image translation using cycle-consistent adversarial networks, in: 2017 IEEE International Conference on Computer Vision, ICCV, IEEE, Venice, 2017, pp. 2242–2251, <http://dx.doi.org/10.1109/ICCV.2017.244>.
- [25] E. Jang, S. Gu, B. Poole, Categorical reparameterization with gumbel-softmax, in: 5th International Conference on Learning Representations, ICLR 2017 - Conference Track Proceedings, Toulon, France, 2017, p. 13, <http://dx.doi.org/10.48550/arXiv.1611.01144>.
- [26] A. Binnering, O. Sorkine-Hornung, SD-pixl: Generating low-resolution quantized imagery via score distillation, in: O. Sorkine-Hornung A. Binnering (Ed.), SIGGRAPH Asia 2024 Conference Papers, New York, USA, 2024, pp. 1–12, <http://dx.doi.org/10.1145/3680528.3687570>.
- [27] Z. Wang, A.C. Bovik, H.R. Sheikh, E.P. Simoncelli, Image quality assessment: From error visibility to structural similarity, IEEE Trans. Image Process. 13 (2004) 600–612, <http://dx.doi.org/10.1109/TIP.2003.819861>.
- [28] X. Mao, Q. Li, H. Xie, R.Y. Lau, Z. Wang, S.P. Smolley, Least squares generative adversarial networks, in: 2017 IEEE International Conference on Computer Vision, ICCV, IEEE, Venice, 2017, pp. 2813–2821, <http://dx.doi.org/10.1109/ICCV.2017.304>.
- [29] L.F. Buzuti, C.E. Thomaz, Fréchet AutoEncoder Distance: A new approach for evaluation of Generative Adversarial Networks, Comput. Vis. Image UnderstAndin. 235 (2023) 1–11, <http://dx.doi.org/10.1016/j.cviu.2023.103768>.
- [30] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna, Rethinking the inception architecture for computer vision, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), vol. 2016, IEEE, 2016, pp. 2818–2826, <http://dx.doi.org/10.1109/CVPR.2016.308>.
- [31] P. Isola, J.-Y. Zhu, T. Zhou, A.A. Efros, Image-to-image translation with conditional adversarial networks, CVPR, in: 2017 IEEE Conference on Computer Vision and Pattern Recognition, vol. 2017, IEEE, Honolulu, 2017, pp. 5967–5976, <http://dx.doi.org/10.1109/CVPR.2017.632>.
- [32] Y. Choi, M. Choi, M. Kim, J.-W. Ha, S. Kim, J. Choo, StarGAN: Unified generative adversarial networks for multi-domain image-to-image translation, in: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, IEEE, Salt Lake City, 2018, pp. 8789–8797, <http://dx.doi.org/10.1109/CVPR.2018.00916>.
- [33] A. Sharma, G. Hamarneh, Missing MRI Pulse Sequence Synthesis Using Multi-Modal Generative Adversarial Network, IEEE Trans. Med. Imaging 39 (2020) 1170–1183, <http://dx.doi.org/10.1109/TMI.2019.2945521>.
- [34] S. Kirkpatrick, C.D. Gelatt, M.P. Vecchi, Optimization by Simulated Annealing, Science 220 (1983) 671–680, <http://dx.doi.org/10.1126/SCIENCE.220.4598.671>.
- [35] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, in: NIPS 2014 Deep Learning Workshop, Montreal, Canada, 2015, p. 9, <http://dx.doi.org/10.48550/arXiv.1503.02531>.
- [36] X. Huang, M.Y. Liu, S. Belongie, J. Kautz, Multimodal unsupervised image-to-image translation, in: Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 11207, LNCS, 2018, pp. 179–196, http://dx.doi.org/10.1007/978-3-030-01219-9_11.
- [37] Y. Zhang, C. Peng, Q. Wang, D. Song, K. Li, S. Kevin Zhou, K.Y. Zhou Zhang, K. Li, S.K. Zhou, Unified multi-modal image synthesis for missing modality imputation, in: IEEE TRANSACTIONS ON MEDICAL IMAGING XX, 2024, pp. 4–18, <http://dx.doi.org/10.1109/TMI.2024.3424785>.
- [38] B. Cao, G. Qi, J. Zhao, P. Zhu, Q. Hu, X. Gao, RTF: Recursive TransFusion for multi-modal image synthesis, IEEE Trans. Image Process. 34 (2025) 1573–1587, <http://dx.doi.org/10.1109/TIP.2025.3541877>.
- [39] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer, High-resolution image synthesis with latent diffusion models, in: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), vol. 2022, IEEE, New Orleans, Louisiana, USA, 2022, pp. 10674–10685, <http://dx.doi.org/10.1109/CVPR52688.2022.01042>.